Frontend Project Structures

Introduction

There is no single “correct” folder structure for every frontend project. The structure you choose depends on your project’s size, complexity, and team. A small portfolio website doesn’t need the same architecture as a large SaaS application.

This guide covers the most commonly used frontend project structures, when to use them, and their advantages.

---

1. Flat Structure

Best For

- Small projects
- Practice applications
- Portfolio websites
- Learning React

Folder Structure

```
src/
├── components/
├── pages/
├── assets/
├── App.tsx
└── main.tsx
```

Advantages

- Very easy to understand
- Quick to set up
- Perfect for beginners
- Less boilerplate

Disadvantages

- Becomes messy as the project grows
- Hard to maintain in large applications

---

2. Type-Based Structure

Best For

- Medium-sized React applications
- Team projects
- Production apps with limited complexity

Folder Structure

```
src/
├── components/
├── hooks/
├── services/
├── utils/
├── context/
├── pages/
├── types/
└── assets/
```

Advantages

- Easy to locate similar files
- Cleaner than a flat structure
- Commonly used in React applications

Disadvantages

- Features become scattered across multiple folders
- Can become difficult to navigate in large projects

---

3. Feature-Based Structure

Best For

- Production applications
- Large React projects
- SaaS products
- Enterprise dashboards

Folder Structure

```
src/
├── features/
│   ├── auth/
│   ├── dashboard/
│   ├── profile/
│   └── products/
├── shared/
├── layouts/
├── routes/
└── App.tsx
```

Example Feature

```
auth/
├── components/
├── hooks/
├── api/
├── pages/
├── types/
└── index.ts
```

Advantages

- Highly scalable
- Easy to maintain
- Better team collaboration
- Related files stay together

Disadvantages

- Slightly harder for beginners
- More planning required

---

4. Atomic Design Structure

Best For

- Design systems
- Component libraries
- Reusable UI
- Large frontend teams

Folder Structure

```
src/
├── components/
│   ├── atoms/
│   ├── molecules/
│   ├── organisms/
│   ├── templates/
│   └── pages/
```

Component Hierarchy

Atoms

- Button
- Input
- Icon

Molecules

- Search Bar
- Product Card

Organisms

- Navbar
- Sidebar
- Footer

Templates

- Page layouts

Pages

- Complete screens

Advantages

- Extremely reusable
- Consistent UI

- Easy to scale

Disadvantages

- Can feel complex for small projects
- Takes time to understand

---

5. Domain-Driven Structure

Best For

- Large SaaS applications
- Enterprise software
- Business-focused products

Folder Structure

```
src/
├── modules/
│   ├── users/
│   ├── orders/
│   ├── payments/
│   └── analytics/
```

Advantages

- Organized around business domains
- Easy to scale
- Great for large teams

Disadvantages

- Overkill for small applications

Which Structure Should You Choose?

Project Type	Recommended Structure
Portfolio	Flat Structure
Small React App	Flat Structure
Medium React App	Type-Based
Production App	Feature-Based
Design System	Atomic Design
Large SaaS	Domain-Driven

Final Recommendation

If you’re just starting out, use a Flat Structure.

For most real-world production React applications, Feature-Based Structure is the best choice because it scales well and keeps related files together.

Choose Atomic Design only if you’re building a reusable component library or design system, and Domain-Driven Structure for very large business applications.